Weekly review: stacks & queues
Select or write the most accurate answer for each question.

When you're finished, click "Send for review" at the top to submit.

1.

A text editor records every user action in a stack:

```
actions = []
actions.append("type A")
actions.append("delete B")
actions.append("bold C")
actions.pop()
actions.pop()
```

What does the stack contain after all operations?

[]

["type A", "delete B"]

["type A"]

["bold C"]

2.

Which of the following correctly implements a queue using deque to process requests in the order they arrive?

```
from collections import deque
q = deque()
q.append("req1")
q.append("req2")
first = q.popleft()
```

```
from collections import deque
q = deque()
q.append("req1")
q.append("req2")
first = q.pop()
```

```
from collections import deque
q = deque()
q.appendleft("req1")
q.appendleft("req2")
first = q.popleft()
```

```
q = []
q.append("req1")
q.append("req2")
first = q.pop(0)
```

3.

A customer support system needs to handle tickets so the first ticket received is always the first one resolved. Which data structure and operation pair correctly achieves this?

Stack — push each new ticket, pop to resolve

Stack — push each new ticket, peek at the bottom to resolve

Queue — enqueue each new ticket, pop the top to resolve

Queue — enqueue each new ticket, dequeue from the front to resolve

4.

Starting with an empty stack, these operations run: PUSH 10 PUSH 20 PUSH 30 POP PUSH 40

What is on the top of the stack at the end?

30

40

20

10

5.

A program processes print jobs. Jobs arrive one after another and must be printed in the exact order they were submitted. A second program keeps browser history so the most recently visited page can be revisited instantly.

Which pair of structures fits these two programs?

Queue for print jobs, Stack for browser history

Stack for print jobs, Queue for browser history

Queue for both

Stack for both

6.

A developer builds an undo system for a text editor using a list, but removes actions with history.pop(0) instead of history.pop().

Both lines remove one item from the list. Explain why the choice of which end to remove from matters here, and what the user would actually experience if the wrong end is used.

an undo system should work from the last edit made (more recent down towards the frst (oldest). Stacks are built oldest to fullest/ pop(0) will incorrectly retireve the firt (oldest) entry thus messing up the timeline of the undo process. history.pop() is thr corect choice a it retireves the rghtmost (last added) item

7.

Two systems are described below:

A browser history feature that lets you press Back to revisit the most recently viewed page.
A customer support ticket system that resolves the oldest open ticket first.
Without naming specific code, explain which data structure principle fits each system and why mixing them up would create a broken experience for users.

browser history is a stack , just like the undo process it is designed to work from most recently addded down towards the oldest entry. The support ticket system is a queue where the first in line is the first helped or generically put, the firt out. mixing them up would let people who just arrived skip to the front of the line and upset everyone who got there earlier. History mixed would harm the ability to systematically track the timeline of pages you visited.

8.

A teammate argues: "I don't need deque for a queue — I can just use a regular list and call pop(0) to remove from the front. It works correctly."

They are right that it works. What would you tell them about why deque is still the better choice, and in what kind of situation would the difference actually show up?

while pop(0) would work its going against the grain as it wr . futureprogrammers tryig to modify the code may not follow the same way you think and iss something, mor os as

the code grows and the details beome burie any lines deep The dequeue is specifally designed or doubble ended retrieval, is an included pythng module thus testd , debugged and used bycountless progrmmers. its also well documented so futur coders should find it easy to folllow

9.

Look at this sequence of stack operations:

PUSH "login"
PUSH "view_profile"
PUSH "edit_photo"
POP
PUSH "save_changes"
POP
POP
Describe the state of the stack after each POP, and explain what the final contents of the stack represent in the context of a user's action history.

["login", "vew profile"] after first pop
["login", "view profile"] again after second pop as "save chages "is added then popped
["login"] after 3rd pop
The contents are down to just the first and oldest entry the users made.

10.

A developer is designing a food delivery app. Orders from customers should be prepared in the sequence they arrive. A driver, however, wants to see the most recently added stop at the top of their list so they can handle it first.

Explain which data structure principle fits each part of the system, and describe what would go wrong for customers or the driver if the wrong principle were applied to either part.

a queue best fits the method of deliveinfg to the first customer to order, first (Fifo). a stack best represents the drivers needs as it shows the most recent first (LiLo)
if implemened the driver would potentially make the first people to order, wait a very long time to get their foodl . The customers food would be cold , the driver would get less or no tip and the owner would suffer as his customers were not handled properly and may not return again

Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Weekly review: stacks & queues

Today
Nice work completing this week, Ron!

How are you feeling about stacks and queues so far?

great

Awesome!

Let me just remind you about the review and how it works.

This review is worth 10% of your final grade for the course.

You can retry it as many times as you need until you pass. Submitted reviews that are on time count toward a bonus at the end.

You'll need to do the work on your own. I can help explain concepts, but I can't give hints or check answers.

Do you feel ready to start the review now?

Yes, I'm ready

Great—once you submit your answers, it takes just a few seconds for the review to be graded.

Good luck, Ron! Give it your best.

To see the assignment, click the "Review" button at the bottom of the chat.

Let me know if you have any questions about the concepts along the way!

Thank you for completing the review. Below is feedback on your submission.

📄 Performance summary
Result: Passed ☑

You demonstrated strong understanding of when to use stacks versus queues, especially for undo systems, browser history, and first-come-first-served processing. Your explanations show clear intuition about ordering and user experience, with particular strength in describing "most recent first" versus "first in line" behavior.
To deepen your skills further, focus on sharpening performance-related reasoning (such as when data structure choices affect speed) and making your scenario explanations concrete and balanced for all parts of a system.

🔍 Detailed Assessment Feedback

Question 2 - For a structure that processes items in the order they arrive, it is important that additions and removals are done at ends that preserve that original arrival order rather than reversing it; when using such a structure in code, carefully track which end represents "first in" to reason about the effect of each operation.
Question 6 - You clearly showed that undo should remove the most recent action, not the oldest one. Your answer explains the correct direction of removal and gives the user-facing consequence that the undo timeline would be wrong if the front were used. That matches the key grading points well.
Question 7 - You correctly matched browser history with the 'most recent first' idea and support tickets with 'first in line first served.' You also explained a real consequence of mixing them up, especially for the ticket system. The browser-history consequence is present but a little less concrete, so this is strong but not fully perfect.
Question 8 - Your response gives good general reasons about readability, maintainability, and using the right tool, but it misses the main point the question was asking about: why one approach becomes slower and when that performance difference matters. To score higher, focus less on style/documentation and more on what happens as the queue gets larger or is used heavily.
Question 9 - You traced the POP results correctly and identified the final remaining item accurately. You also showed that earlier actions remain after later ones are removed. The only reason this isn't full credit is that your final explanation is a bit brief and could more clearly connect the remaining stack to the idea of actions that have not been undone.
Question 10 - You identified the right principle for customer orders and for the driver's list, which shows good understanding. Your explanation of what goes wrong is strongest for the customer-order side. However, the driver-side consequence is not explained clearly enough, so the answer feels somewhat one-sided. To score higher, make the failure case concrete for both parts of the system.
Well done on passing this review; your grasp of stack and queue behavior is solid, and with a bit more attention to performance implications and balanced scenario explanations, you will be in an even stronger position going forward.

Thank you for your feedback!